

Caching

1 The cache structure

1. A 64KiB, direct mapped cache has 16 byte blocks. You may assume that the memory is byte addressed (so 1 word = 1 byte). If addresses are 32 bits long, how many bits are provided for the tag, index, and offset in this cache?
2. Adjust your answer if the cache is 4-way set associative rather than direct.
3. How many bits do we need for the index if the cache is fully associative? Explain your answer.

2 Hit/miss in a 2-way LRU cache

An 8 byte, 2-way set associative cache (with LRU replacement) with 2 byte blocks receives requests for the following (binary) addresses:

0110,0000,0010,0001,0011,0100,1001,0000,1010,1111,0111.

After each access, provide the contents of the cache by filling in the attached table, for each set you must list the tags present.

Address	Set 0	Set 1	Hit/Miss
0110	empty/empty	01/empty	Hit/Miss
0000	/	/	Hit/Miss
0010	/	/	Hit/Miss
0001	/	/	Hit/Miss
0011	/	/	Hit/Miss
0100	/	/	Hit/Miss
1001	/	/	Hit/Miss
0000	/	/	Hit/Miss
1010	/	/	Hit/Miss
1111	/	/	Hit/Miss
0111	/	/	Hit/Miss

3 Cache hit rate

3M decides to make Post-Its by printing yellow squares on white paper, where for each point in the square the CMYK (cyan, magenta, yellow, black) value has to be filled in. 3M hires you to look at three implementations on a machine with a 2048-byte direct mapped data cache with 32 byte blocks/lines. You are given the following definitions:

```
struct point_color {
    int c;
    int m;
```

```

int y;
int k;
};
struct point_color square[16][16];
register int i, j;

```

Suppose further that

- `sizeof(int) = 4`,
- square starts at memory address 0,
- the cache is initially empty,
- the only memory accesses are to the entries of the array square. The variables `i` and `j` are stored in the registers.
- the memory is addressed 4-byte, so 1 int = 1 word.

What is the percentage of writes in the following code that result in a hit?

Method 1:

```

for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[i][j].c = 0;
        square[i][j].m = 0;
        square[i][j].y = 1;
        square[i][j].k = 0;
    }
}

```

Method 2:

```

for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].y = 1;
        square[j][i].k = 0;
    }
}

```

Method 3:

```

for (i=0; i<16; i++) {
    for (j=0; j<16; j++) {
        square[j][i].y = 1;
    }
}

```

```

for (i=0; i<16; i++){
    for (j=0; j<16; j++) {
        square[j][i].c = 0;
        square[j][i].m = 0;
        square[j][i].k = 0;
    }
}

```

4 Cache replacement algorithms

For the tree-based and MRU-based pseudo LRU algorithm, check the following two things and prove these properties if necessary:

1. Suppose that w is the most recently used word is, for $i \neq 1$ to the set size k . LRU can only contain one word w_k be replaced. Which words can be replaced if we use a pseudo LRU?
2. Suppose a certain block B (di, a certain line) is no longer used from a certain point on. How long can block B then still remain in the cache? In other words: how many replacements can B maximum 'survival'?

5 Access times and hit rates

Let T_1 represent the access time if a word in the L_1 cache is located. If the word is only in the L_2 cache is present and not in the L_1 cache, then there is a time T_2 necessary to convey the word to the L_1 cache. Finally, there is a time T_M needed to transfer a word from the main memory to the L_2 cache to move. Suppose that $T_1 = 20$ nsec, $T_2 = 100$ nsec and $T_M = 400$ nsec.

1. If we respectively have a hit rate of .9 and .5 in the L_1 and L_2 cache, what is the average access time?
2. If the hit rate in the L_2 cache is .6, what is the hit rate in the L_1 cache if the average access time is 33 nsec?

6 Direct mapping versus set-associative mapping with random replacement

Consider the following piece of code from the theory slides.

```
float a[1024], b[1024]; for
(i=0; i<1024; i++)
    sum += a[i] * b[i];
```

Also suppose:

- Cache size: 4 KByte, Block size: 32 byte, Word size: 4 byte
- Float: 4 byte
- b sits right after a in memory
- i and sum are registers

1. What is the hit rate if we assume direct mapping?
2. If we assume a 4-way set-associative memory with random replacement, calculate the (approximate) hit rate.
3. Adjust your answer to the previous 2 questions, if you know that both arrays contain only 256 elements each. Do the same for 1536.

7 Benchmarking

Draw a graph illustrating a cache benchmark as described on p31 of the theory course. We consider a system that only has an L_1 cache; any TLB and paging effects do not need to be shown. The size of the L_1 cache is 32 KByte, a cache line contains 64 bytes and 8-way associativity is assumed. A hit has an access time of 50 ns, a miss costs an additional 500 ns. Make the graph, with the stride size (logarithmic scale) on the x-axis and the access time on the y-axis, for array sizes from 4 to 512 Kbyte.

Memory Management

1 Buddy system

1. Consider a buddy system of 1Mbyte in size. Graphically represent the state of the memory after the following actions, if you know that the memory is initially empty.

- (a) Request A = 70 Kbyte
- (b) Request B = 35 Kbyte
- (c) Request C = 80 Kbyte
- (d) Return A
- (e) Request D = 60 Kbyte
- (f) Return B
- (g) Return D

2. In the standard buddy system, processes are ultimately allocated memory blocks of powers of 2. Suppose we want to use a Fibonacci sequence instead.

- Explain the operation of such a system using a short example. (tip: assume that the total memory size is a (large) Fibonacci number)
- Does this system perform better or worse than the standard buddy system in terms of internal fragmentation? Why (not)?

Paging

1 Size of the page tables of a process

Suppose we have a 32-bit machine with virtual addresses split into 4 parts of 10, 8, 6, and 8 bits respectively. We handle these addresses in a 3 level page table system, where the first 10 bits are used for level 1, the next 8 bits for level 2, etc. A single page table entry is 4 bytes in size. The system is byte addressed.

1. What is the page size in this system?
2. Suppose a process requires 256K of memory (starting from address0),How much space do the page tables of this process take up?
3. Suppose a process has a code segment of 48K from (virtual) address0x10000000, a 600K data segment from address0x80000000and a stack segment of 64Kfrom address 0xF0000000.How much space does the page table of this process require?

2 Determine the number of levels in a multilevel paging system

Suppose a machine has virtual addresses consisting of 36 bits and the pages are 8K in size. A single page entry is 8 bytes long.

1. How many pages does the virtual address space consist of?
2. Suppose an average process is 8 Gbytes in size. How many levels are most suitable if we work with a multilevel page table (consider 1 to 3 levels)? Weigh the pros and cons against each other. For the 2-level system, choose the size of a level 2 page table such that it fits exactly in a single page. For three levels, take a more or less equal division for each of the fields.

3 Count the number of TLB misses

Consider the following piece of code. Suppose that the binary code fits on a single page and that the stack also fits on a single page. You may also assume that when the function is called, the stack and code page are already in the TLB cache. Suppose further that an integer is 4 bytes long and that a page contains 4096 bytes.

```
int a[1024][1024], b[1024][1024], c[1024][1024];
multiply() {
    unsigned i,j,k;
    for(i=0; i< 1024; i++)
        for(j=0; j< 1024; j++)
            for(k=0; k< 1024; k++)
                c[i][j]+=a[i][k]*b[k][j];
}
```

Given that the TLB cache contains only 8 entries and that it uses an LRU replacement policy, what is the number of TLB misses when executing this code?

4 Demand paging: good or bad

Which of the following programming techniques works? *Good* in combination with a demand paged system.

1. Stack
2. Hash table
3. Sequential search method
4. Binary search method
5. Pure code
6. Vector operations (e.g. vector multiplication)
7. Indirect addresses

5 Useful Interventions for Demand Paging

Suppose we take some statistics on a demand paged system. We see that the CPU is used at 20%, the disk with the pages at 97.3% and the other I/O devices at only 5%. What interventions do you think would be effective to improve the system?

1. Install a faster CPU
2. Install a larger paging disk
3. Increase the degree of multiprogramming
4. Reduce the degree of multiprogramming
5. Install more memory
6. Install a faster hard disk
7. Allow prepaging
8. Increase page size

6 Inverted versus standard page tables

If virtual addresses are 64 bits long, page size is 1Kbyte, physical memory is 64Mbyte, and a PTE takes up two bytes, what is the total size of the inverted and standard page table (assuming every byte is addressed)? First, estimate how many bytes an inverted page table entry will take up, by determining the minimum number of fields it must contain, and how many bits each field must contain.

7 Inverted page table

Consider a system with 16KB of memory, pages of 1KB in size, an inverted page table, and a full-associative TLB with 2 entries and an LRU replacement policy. The hash function used is $f(PID, VPN) = (10 * PID + VPN) \bmod 24$. The system needs 0.5ns to read the TLB, 100ns to read the main memory and 10ms to access the hard disk. Fill in the table below with page requests using the given hash anchor table and the inverted page table (IPT). In the IPT, "T" is used as a symbol to indicate that there is no next page table entry (PTE).

In the TLB column you display the contents of the TLB after the translation has been done. For this, use the notation "(VPN, Frame number)" for each line in the cache, use "-" if the line in question is empty. In the columns called "Frame number", "TLB hits" and "Page fault" you display respectively the physical frame number, whether there was a TLB hit during the translation and whether it resulted in a page fault or not. Finally, in the last column you fill in the time that was needed to perform the translation. We assume that the buffer can be updated in parallel after the translation and the execution of the next instruction, so you do not need to calculate any time for this. In the case of possible page faults you do not need to adjust the data structures, but you do need to provide the time that was needed to find out that there is a page fault.

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
Value	7	3	12	15	13	6	2	4	0	14	8	5	15	11	10	8	5	7	9	10	12	6	13	1

Table 1: Hash anchor table

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
PID	0	3	2	3	0	8	0	2	2	0	8	6	0	2	2	4
VPN	8	17	8	4	1	3	5	4	2	3	13	1	2	32	0	4
Next PTE	2	T	T	4	T	T	10	12	3	T	0	9	14	8	T	11

Table 2: Inverted Page table

PID	VPN	TLB	Frame number	TLB hits	Page fault	Latency (ns)
0	1			Hit/Miss	Yes/No	
0	2			Hit/Miss	Yes/No	
0	1			Hit/Miss	Yes/No	
0	3			Hit/Miss	Yes/No	
8	3			Hit/Miss	Yes/No	
0	3			Hit/Miss	Yes/No	
2	2			Hit/Miss	Yes/No	
2	1			Hit/Miss	Yes/No	
8	13			Hit/Miss	Yes/No	
8	3			Hit/Miss	Yes/No	

8 Count the number of page faults

Consider the following sequence of pages:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6.

How many page faults will occur if this process is over 1,2, . . . ,6 or 7 frames. Check this for the following three replacement algorithms: LRU, FIFO and the MIN algorithm. Initially, there are no pages in memory.

9 Segmented FIFO

True or false?

1. All pages in L1 were requested more recently than those in L2.

2. Pages in L1 are in LRU order.
3. Pages in L2 are in LRU order.
4. $L2/-1$ Most recently used pages are safe (cannot be replaced).
5. $L2$ /most recently used pages are safe.
6. A segmented FIFO system with $L1 \neq L2 \neq 2$ is equivalent to a tree-based pseudo-LRU system with 4 pages.

10 Benchmarking

Draw a graph illustrating a cache benchmark as described on p31 of the theory course. Consider a system that has an L1 cache and a TLB cache. The size of the L1 cache is 256 KByte, a cache line contains 32 bytes and direct mapping is assumed. An L1 cache access takes 250ns, an L1 cache miss adds 500ns, a TLB miss adds another 1000ns. The TLB cache is 4-way associative, contains 128 entries, and the page size is 8192 bytes.

Create the graph, with stride size (logarithmic scale) on the x-axis and access time on the y-axis, for array sizes from 4 Kbytes to 4096 Kbytes.

Uniprocessor scheduling

1 A variation on highest response ratio next

A variant of HRRN, called minimax response ratio, aims to minimize the maximum response ratio. Think of a scheduling policy that satisfies this. Take inspiration from the example in Figure 1; suppose we have to make a scheduling decision at time t . In what order should tasks be t_1, t_2 and t_3 will be executed (from time t), so that the maximum response ratio $\max_s ((w + p_s)/p_s)$ is as small as possible?

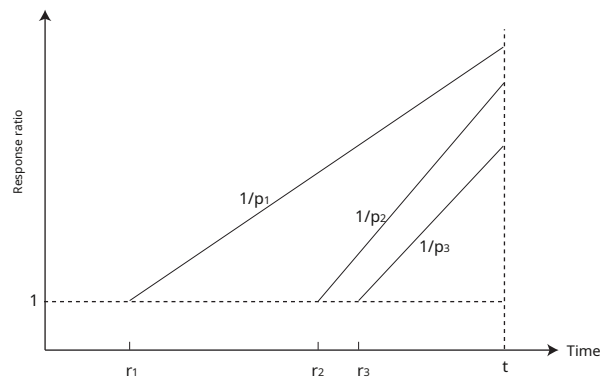


Figure 1: Response ratio as a function of time

2 Rate monotonic scheduling

As already mentioned in the course, it can be demonstrated that RMS will always respect the deadlines τ_i if $YOU = \sum_{j=1}^n p_j / T_j \leq n(2^{1/n} - 1)$

1. Provide an example of jobs (at least 2 jobs) that meet their deadlines with RMS, and YOU is equal to 1.
2. Give an example of jobs that do NOT meet their deadlines with RMS, and YOU is at most 0.95.
3. Give an example of jobs that do NOT meet their deadlines with RMS, and YOU is at most 0.84.

Disk Management

1 Random seek

Consider a hard disk with N tracks. Now assume that the requested sectors are randomly distributed over all tracks and independent of each other. Now calculate the average number of tracks that the disk head must travel between two requests, as follows:

1. What is the probability that the disk head is in position i state and position j must?
2. How many tracks must be completed from i to j ?
3. Combine 1 and 2 by using all possible combinations of i and j to investigate and work out.
4. Given that N is large, give an approximation for the number of tracks to be covered, relatively compared to the total number of tracks.

2 Scheduling algorithms

Suppose we have an IO controller that uses the SSTF algorithm, and the disk head is located on the center track of the disk.

1. Now give a series of 8 requests, where the disk head must pass over the middle of the disk between each request. Give the minimal (in terms of total tracks to be covered) solution for this problem. (You may write the requests relative to the middle track; in other words, if the middle track is 512, you may indicate track 511 as -1.)
2. Calculate the total number of track changes for this request pattern.
3. Run the same pattern through a LOOK algorithm, and calculate the number of track changes for this. Compare.

3 RAID efficient?

Given that you have 12 hard drives of 40GB each, consider for the following RAID configurations, how many GB you can ultimately fit on this RAID system?

- RAID 0+1
- RAID 1+0
- RAID 0+5 (all 3 possible splits)
- RAID 5+0 (all 3 possible splits)

4 RAID: robust?

Given that you have 12 hard disks, calculate for the following RAID configurations, how many disks can fail at most, without the system failing (not necessarily irreparably). Then calculate the probability that the system remains operational, when two hard disks fail.

- RAID 0+1
- RAID 1+0
- RAID 0+5 (all 3 possible splits)
- RAID 5+0 (all 3 possible splits)

Concurrency

1 Producer/consumer problem

Give a possible implementation for the producer/consumer problem with semaphores, assuming that the buffer into which the data is written is now finite; in other words, if the producer wants to write an item and the buffer is full, it must wait until a space becomes free again.

2 Concurrency

Consider the following piece of code:

```
const int n = 50; int
tally;
void total() {
    int count;
    for (count = 1; count <= n; count++) {
        tally=tally+1;
    }
}

void main() {
    tally = 0;
    parbegin(total(),total()); // execute in parallel cout << tally
    << endl;
}
```

1. What is the minimum and maximum value that the variable *tally* can assume, at the end of the program?
2. Suppose that there M parallel instances of *total* are executed, what changes these boundaries?

3 Paper review

Reviewing papers is one of the many activities of researchers. The purpose of this is to “check” a paper, among other things for correctness, before it can be published in a journal or presented at a conference. Below *comment* was submitted in 1965 to the highly regarded journal Communications of the ACM. Review this comment, in Figure 2, for correctness. (Hint: look for an execution sequence in which two processes access the critical section simultaneously)

✓ Comments on a Problem in Concurrent Programming Control

Dear Editor:

I would like to comment on Mr. Dijkstra's solution [Solution of a problem in concurrent programming control. *Comm ACM* 8 (Sept. 1965), 569] to a messy problem that is hardly academic. We are using it now on a multiple computer complex.

When there are only two computers, the algorithm may be simplified to the following:

```
Boolean array  $b(0; 1)$  integer  $k, i, j$ ,  
comment This is the program for computer  $i$ , which may be  
either 0 or 1, computer  $j \neq i$  is the other one, 1 or 0;  
 $C0$ :  $b(i) := \text{false}$ ;  
 $C1$ : if  $k \neq i$  then begin  
 $C2$ : if not  $b(j)$  then go to  $C2$ ;  
    else  $k := i$ ; go to  $C1$  end;  
    else critical section;  
     $b(i) := \text{true}$ ;  
    remainder of program;  
    go to  $C0$ ;  
    end
```

Mr. Dijkstra has come up with a clever solution to a really practical problem.

HARRIS HYMAN
Munitype
New York, New York

Figure 2: Comment submitted to Communications of the ACM.

4 Binary semaphores

```
void semWait(semaphore s) {
    semWaitB(mutex);
    s--;
    if (s < 0) {
        semSignalB(mutex);
        semWaitB(delay);
    } else
        semSignalB(mutex);
}

void semSignal(semaphore s) {
    semWaitB(mutex);
    s++;
    if (s <= 0)
        semSignalB(delay);
    semSignalB(mutex);
}
```

This code could represent a possible implementation, which *counting semaphores* realizes, using binary semaphores. Unfortunately, this code can still cause a problem. Find this problem, and adjust the code so that you get a correct solution.

5 Mutual exclusion

Given the code below, answer the following questions.

- Is mutual exclusion guaranteed for the critical section? Explain.
- Is it possible to get into a deadlock? Explain.
- Is starvation possible? Explain.

```
1  global variables : intturn
2  =-1;
3  boolbusy =false ;
4
5  threadi :_
6      while (turn != i ){
7          while (busy ) {
8              turn = i ;
9          }
10         busy =true ;
11     }
12 }
13
14 <critical section>
15
16 busy =false ;
```

6 Model of an amusement park

Consider an amusement park attraction consisting of a ride in a 1-person car through a haunted house. In total, the attraction has n cars on a fixed track (the cars cannot overtake each other) and to save energy, only the full cars drive around; the others are waiting in the station for passengers. The m visitors of the amusement park walk around the amusement park for a while, arrive at the haunted house, wait for a car in a FIFO queue if necessary, get into the first available car, enjoy the attraction, get out and then complete their visit to the park.

If we model both the visitors and the carts as processes that need to synchronize (using semaphores), it is essential to avoid

- A cart can leave even if it is not yet full.
- A cart can depart while a passenger is getting on or off that cart.
- Passengers disembark too early, i.e. they are not allowed to leave the carriage during the ride.

Given the global variables:

`cars_avail` = semaphore initialized on 0; `passenger_safe[]` = array of n semaphores initialized on 0; `ride_done[]` = array of n semaphores initialized on 0;

Full code for the carts:

```
car_j:
while (turned_on) {
    signal(cars_avail)
    <pick up human i>
    wait(passenger_safe[j]) <drive
    around with human i>
    signal(ride_done[j])
    <drop off human i>
    wait(passengers_safe[j])
}
```

Partial code for the passengers:

```
human_i:
...
<wander around in
park> ...
<get into car j> ...

<get scared>
...
<get out of car j> ...

<complete visit to park> ...
```


- Complete the code of the visitors. Only use the known semaphore operations.
- What needs to be changed if the carts can carry k people? Give the modified code for both the carts and the visitors. We assume that passengers do not communicate with each other, and cannot find out how many passengers are already in the cart. (2pt)
- Are there boundary conditions on k and m ? Explain. (1pt)

Cache coherency

1 MESI protocol

A system has two processors, each with a fully associative cache of 4 lines. Both caches use the tree-based replacement algorithm, are initially empty and the processors use the MESI protocol. Complete the following table. The requests state *R* for *read* and *W* for *write*.

Memory access		Cache 1	Cache 2	Comments
Proc 1	Proc 2	---- / IIII	---- / IIII	
R1		1 --- / EIII	---- / IIII	
R2		1 - 2 - / EIEI	---- / IIII	
	W3			
	R4			
W5				
R6				
	R2			
W2				
	W1			
	R3			
R1				
	R6			
W7				
	R1			
	W7			
R1				
R6				

Multiprocessor scheduling

1 List Scheduling Algorithm

Give a (general) example where LS is also effectively $2 - 1/m$ times worse than an optimal algorithm, as a function of the maximum completion time; in other words, show that this upper bound cannot be improved. First, try to find an example for $m=2,3$, then give a general example.

2 Longest Processing Time First

Give a (general) example where LPT is also effective $4/3 - 1/3m$ times worse than an optimal algorithm, as a function of the maximum completion time; in other words, show that this upper bound cannot be improved. First, try to find an example for $m=2,3,4$, then give a general example.